

BriefPilot — Technology Stack Decision Document

Version 1.0 · July 2026 · Pricing verified against public sources July 2026 (links inline; re-verify before committing spend — this market reprices quarterly)

1. Executive Summary

The recommended stack:

Layer	Choice	Monthly cost at MVP
Frontend	Next.js (App Router, TypeScript)	€0
Backend	Python + FastAPI + Pydantic	€0
OCR	Abstraction layer; Azure Document Intelligence Read (S0, EU region) as primary; Mistral OCR as EU-native alternative; Tesseract as free dev fallback	~€0-3
LLM	Claude Haiku 4.5 via thin model-agnostic client; Sonnet-class escalation path	~€5-10
Orchestration	None. Native SDK calls + Pydantic schemas	€0
Database	PostgreSQL 16 in Docker on the VPS	€0
File storage	VPS local volume (24 h auto-purge)	€0
Auth	None in MVP; Better Auth when accounts arrive	€0
Hosting	Hetzner CX22 VPS, Falkenstein (Germany) + Docker Compose + Caddy	~€4-5
Observability	Sentry free tier (EU data residency) + Uptime Kuma + structured logs	€0
CI/CD	GitHub Actions (free, public repo)	€0

Total MVP run rate: ~€10-15/month, dominated by the VPS and per-letter AI costs. Unit economics: **€0.03-0.06 per analyzed letter** (OCR + LLM), which means a future €2-5 per-case price carries a 97%+ gross margin on AI costs.

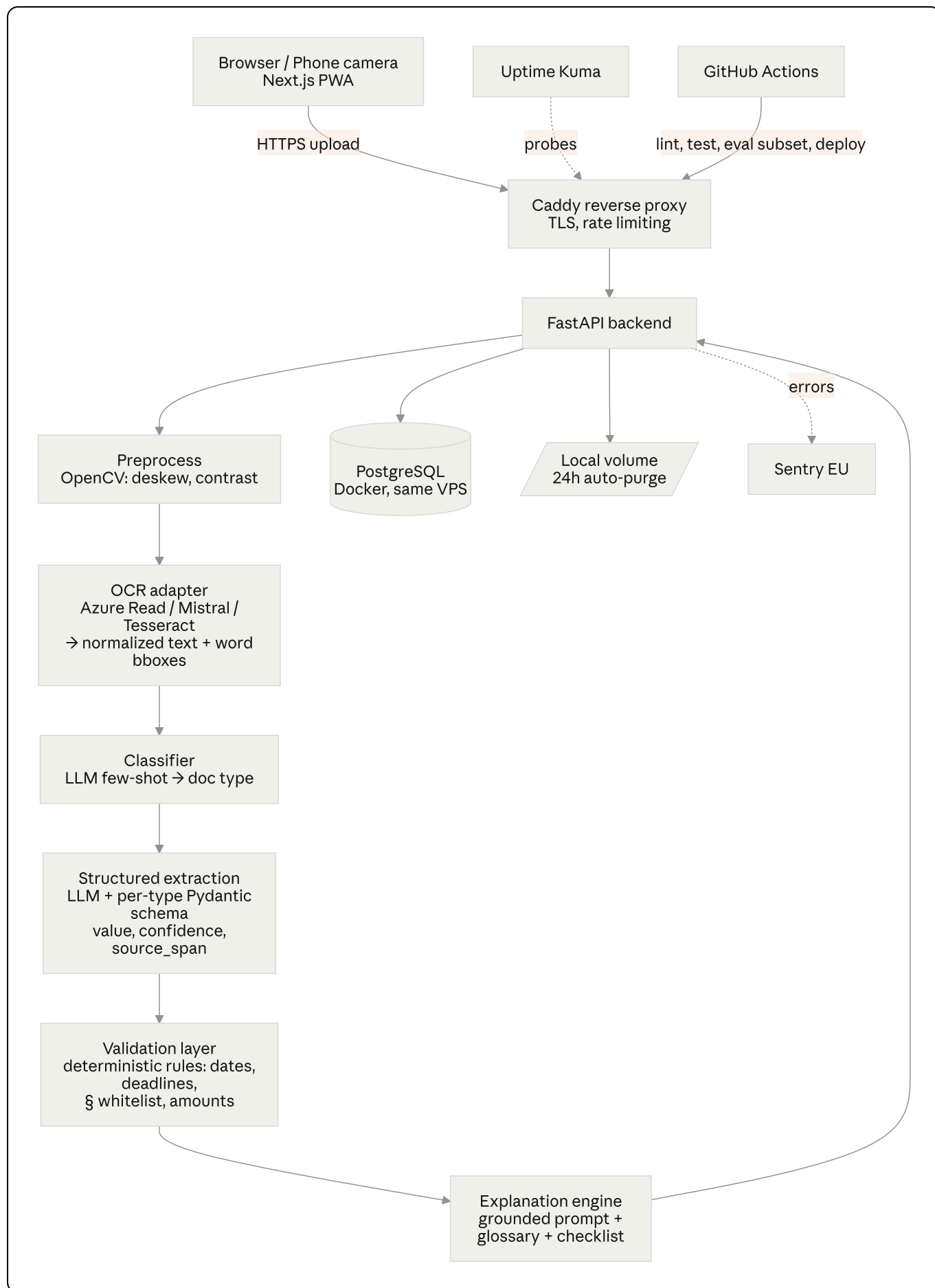
Three principles drove every choice:

1. **"Hosted in Germany" is a feature, not a constraint.** A Hetzner box in Falkenstein plus EU-region AI endpoints turns the GDPR requirement into the product's trust

story and the portfolio's differentiator. Every layer was filtered through "can I honestly write this on the privacy page?"

2. **Boring infrastructure, interesting pipeline.** Zero engineering novelty is spent on hosting, auth, or frameworks. All complexity budget goes to the document pipeline (OCR normalization, validation, eval) — the part hiring managers actually evaluate.
 3. **Abstraction at the two volatile seams only.** OCR provider and LLM provider sit behind thin internal interfaces because both markets reprice and leapfrog quarterly. Everything else is used directly — premature abstraction is how solo projects die.
-

2. Architecture



Why each component exists: Caddy terminates TLS and rate-limits before anything touches Python. Preprocessing exists because phone photos are the dominant real-world input and OCR quality is the pipeline's ceiling. The OCR adapter normalizes every provider

into one `{text, page, bbox, confidence}` schema so the highlight overlay and the eval fixtures survive a provider swap. Classification routes to per-type schemas because a Finanzamt Bescheid and a rental notice share no fields. The validation layer is the anti-hallucination firewall — deterministic code, not another LLM. Postgres and local storage live on the same box because at MVP volume, network hops between managed services add cost and GDPR surface for zero benefit. Sentry and Uptime Kuma exist because a portfolio project that silently 500s during a recruiter's visit was wasted work.

3. Layer-by-Layer Decisions

3.1 Frontend — Next.js

	React (Vite)	Next.js	Remix	Vue/Nuxt	SvelteKit
Fit for this product	Fine	Best	Good	Good	Good
Hiring-market value (DE/EU 2026)	High	Highest	Niche	Medium	Growing but niche
Ecosystem for file upload, PWA, SSR landing/SEO	Assemble yourself	Built-in	Good	Good	Good
AI-assisted dev quality (Claude Code/Cursor output)	High	Highest (most training data)	Medium	Medium	Medium

Choice: Next.js with TypeScript. Not because it's technically superior for a two-page app — it's arguably overkill — but because this project has two customers: users and hiring managers. Next.js maximizes the second without penalizing the first, gives a free SSR landing page for SEO, and produces the best AI-pair-programming output, which matters at 6–8 solo hours/day. Rejected: SvelteKit (better DX, worse hiring signal in the German market), Remix (merging into React Router reduced its standalone signal), Vue (fine, but splits the portfolio story — the backend is already Python; one "mainstream" choice per layer).

Cost: €0.

3.2 Backend — FastAPI

	Express	NestJS	Fastify	FastAPI	Django
AI/ML ecosystem	Weak	Weak	Weak	Native (every OCR/CV/eval lib is Python-first)	Native
Schema-first extraction	Zod (fine)	Class-validator	Zod	Pydantic — the extraction schemas ARE the API layer	DRF serializers
Async pipeline fit	Good	Good	Good	Good	Poor (heavier)
Weight for a solo MVP	Light	Heavy	Light	Light	Heavy

Choice: FastAPI. The killer argument is Pydantic: BriefPilot's core artifact is schema-constrained LLM extraction, and Pydantic models serve simultaneously as the extraction contract, the validation layer, the API response types, and the eval fixture format. One type system, four jobs. OpenCV/Pillow preprocessing and the eval harness are also Python-native. Rejected: NestJS (enterprise ceremony a solo dev pays for and never collects on), Django (batteries this project doesn't need — no admin, no ORM-heavy CRUD), Node options (would force a JS bridge to every document-processing library).

Cost: €0. Learning curve: minimal beyond Python itself; FastAPI is the closest thing Python has to self-documenting.

3.3 OCR — the most consequential decision in the stack

Requirements recap: German Amtsdeutsch, phone photos AND scanned PDFs, tables and stamps, **word-level bounding boxes** (the source-highlight feature depends on them), EU processing.

Option	Price / 1,000 pages	Bboxes	German quality	EU story	Verdict
Tesseract (self-host)	€0 (CPU only)	Yes (hOCR)	Good on clean scans, poor on phone photos/tables	Perfect (your box)	Free dev fallback + offline fixture testing
PaddleOCR (self-host)	€0 + GPU ops	Yes	Strong, esp. layouts	Perfect	Better than Tesseract but adds ops weight a solo dev shouldn't carry at MVP
Azure Doc Intelligence — Read (S0)	~\$1.50	Yes, word-level	Strong	EU regions (e.g. Germany West Central) + Microsoft DPA	Primary recommendation
Azure DI — Layout	~\$10	Yes + tables/structure	Strong	Same	Overkill at 6.7× the price; letters are not invoices. Upgrade path only
Mistral OCR (3/4)	~\$2-4 standard, ~\$1-2 batch	Structured annotations/bbox (verify granularity in bake-off)	Strong, markdown + table reconstruction	French company, EU-native — best story	Primary alternative; also available via Vertex <u>europa- west4</u>
Google Document AI	~\$1.5-10 by processor	Yes	Strong	EU region available	Fine, no advantage over Azure here; more setup friction
AWS Textract	\$1.50 basic; \$15+ with tables	Yes	Good	EU regions	Only rational inside an existing AWS estate; table pricing is punitive

Free-tier trap to know: Azure's F0 free tier gives 500 pages/month but **processes only the first 2 pages of any request** (silently dropping the rest) with a 4 MB cap. Fine for development against 1-2-page letters; a silent-data-loss bug waiting to happen in production. Budget the S0 tier from launch — at MVP volume it costs cents.

Recommendation:

- **Best free:** Tesseract behind the adapter, used for local dev and CI fixture tests so the paid API isn't hit on every test run.
- **Best paid:** Azure DI Read S0 in a German region — cheapest bbox-bearing managed OCR, strong on German, clean DPA story.
- **Best long-term architecture:** the adapter itself. Normalize all providers to one internal schema on day one; run the Day-3 bake-off from the execution plan between Azure Read and Mistral OCR on your real fixture letters and let measured accuracy on *your* documents — not blog benchmarks — break the tie. If Mistral's bbox granularity satisfies the highlight feature, its EU-native identity is worth a small price premium for the trust narrative.

Cost at MVP: ~900 pages/month ~ €1.30 (Azure Read) to €3.60 (Mistral standard).

3.4 LLM — Claude Haiku 4.5, behind a thin client

Current pricing landscape (per 1M tokens, mid-2026): Claude Haiku 4.5 \$1/\$5 · Claude Sonnet 4.6 \$3/\$15 · Gemini 2.5 Flash \$0.30/\$2.50 · GPT-5.4 mini ~\$0.25-0.75/\$2-4.50 · Mistral Large 3 \$0.50/\$1.50 · DeepSeek V4 \$0.30/\$0.50. Batch APIs generally halve costs; Anthropic's prompt caching discounts repeated system prompts heavily — relevant because BriefPilot's extraction prompts are long and identical across requests.

Need	Pick	Why
Cheapest viable MVP	Gemini 2.5 Flash or DeepSeek V4	Rock-bottom tokens; Flash adds a generous free tier for dev. DeepSeek's EU/GDPR story is disqualifying for production here
Best quality	Claude Sonnet 4.6 / Sonnet 5	Strongest instruction-following on schema-constrained extraction; reserve for an escalation tier
Best EU/GDPR story	Mistral (Large 3 / Medium)	EU company, EU residency, open weights — unmatched narrative; slightly behind on hardest extraction
Recommended default	Claude Haiku 4.5	Near-Sonnet reliability on structured, multi-step instructions at 1/3 the price; 200K context covers any letter; prompt caching cuts the repeated-system-prompt cost ~90%; strong JSON discipline reduces parser retries — which are a real hidden cost

Architecture rule: one internal `(llm_client.complete(prompt, schema))` wrapper, model name in config, prompts versioned in the repo, and the eval harness re-run on any model

change. That makes "migration path" a config edit plus a 10-minute eval run — the correct answer to vendor risk at this scale. A pragmatic router is the natural upgrade: Flash-class for classification (cheap, easy task), Haiku for extraction, Sonnet-class retry when field confidence comes back low.

Data protection: use API tiers where inputs aren't used for training (standard for Anthropic's API; verify current terms), enable EU processing where offered (Vertex/Bedrock EU regions host Claude and Mistral), disclose the subprocessor on the privacy page. Self-hosted open-source (Ollama/vLLM) is the only true "no third party" answer but demands a GPU (~€100+/month rented) and quality babysitting — the wrong trade until either scale or an enterprise customer demands it. Keep it as the documented endgame, not the MVP.

Cost per letter (3 pages ≈ 4K doc tokens; classify + extract + explain ≈ 15K in / 4K out): Haiku ≈ **\$0.035**; with caching and a Flash classifier ≈ **\$0.02**. MVP month (300 letters): ~**€6-10**.

3.5 Document Processing — no framework

LangChain and LlamaIndex earn their complexity when you need dozens of integrations, agentic branching, or a RAG index over large corpora. BriefPilot's pipeline is a **linear, five-stage assembly line** with deterministic checkpoints. Wrapping that in a framework adds a dependency-churn tax, obscures the very pipeline the portfolio is supposed to showcase, and — the part nobody says out loud — reads as "wrapper project" to senior engineers. Native Anthropic/OpenAI SDK calls + Pydantic + ~200 lines of your own orchestration code IS the demonstration of engineering skill. The Vercel AI SDK is a fine frontend-streaming convenience if chat features arrive later; it is not a pipeline tool. The curated legal-reference lookup at MVP is a small, versioned JSON/SQLite table — not a vector database. Add pgvector inside Postgres if semantic retrieval is ever genuinely needed; never a separate vector-DB service at this scale.

3.6 Database — PostgreSQL, self-hosted first

	SQLite	Postgres (Docker on VPS)	Supabase	Neon	MongoDB
MVP fit	Honestly sufficient	Right	Fine	Fine	Wrong shape
Cost	€0	€0 (on existing VPS)	Free tier (500 MB, pauses after inactivity)	Free tier (autosuspend)	Atlas free tier
GDPR	Perfect	Perfect (German soil)	EU region selectable	EU region selectable	EU region selectable
Portfolio signal	"Toy?" (unfairly, but real)	Standard	"Used a BaaS"	Fine	Off-market for relational data

Choice: Postgres 16 in Docker Compose on the Hetzner box, with nightly `pg_dump` to a second location. Extraction results are relational and typed; JSONB columns absorb the flexible per-type fields. SQLite would genuinely work for a session-based no-auth MVP, but Postgres costs the same €0 here, removes a migration, and is the answer German backend interviews expect. Supabase is the right *managed* migration path when accounts/realtime arrive — its free-tier project pausing is why it's not the primary. Rejected: MongoDB (schema-validated extraction is the opposite of schemaless), Neon (nice serverless Postgres, but a second vendor for no MVP gain).

3.7 File Storage — local volume now, S3-compatible later

Documents live ≤24 hours by design (auto-purge is a privacy feature). A 40 GB NVMe VPS volume handles that indefinitely at MVP volume for €0 extra. When persistent document history ships (post-MVP accounts), move behind an S3-compatible interface: **Hetzner Object Storage** (EU, ~€5/month baseline) keeps the everything-in-Germany story; **Cloudflare R2** (10 GB free, zero egress fees) is the cost-optimal alternative if the trust narrative can absorb a US-company subprocessor with EU jurisdiction settings. Code against the S3 API via a storage adapter from day one (boto3-compatible), so "local vs. object storage" is a config flag. Rejected as MVP: S3 (egress pricing + no free EU story), Backblaze B2 (cheap, but US-centric), Supabase Storage (only makes sense if Supabase becomes the DB).

3.8 Authentication — none, then Better Auth

The MVP is deliberately account-free (see execution plan): session-based analysis, one-click delete, auto-purge. This is a trust feature, saves ~2 days, and deletes an entire GDPR

surface (no consent flows, no credential storage, no processor agreements for an auth vendor).

When accounts arrive: **Better Auth** — open source, runs inside your own backend, credentials in your own Postgres on German soil, €0 forever, and "I own my auth data path" is a stronger interview line than "I integrated Clerk." Clerk (10K MAU free) is the fastest DX but adds a US subprocessor to the most sensitive product category imaginable; Supabase Auth only if Supabase wins the DB migration; Auth.js is fine but Better Auth has overtaken it on DX and momentum; Firebase Auth drags in the Google ecosystem for nothing this product needs.

3.9 Deployment — Hetzner VPS + Docker Compose + Caddy

	Vercel	Railway	Render	Fly.io	Hetzner VPS	Coolify on Hetzner
Real monthly cost (MVP)	Free tier, but Python backend doesn't fit; functions model wrong for long OCR jobs	~\$5+ usage-based	Free tier sleeps; ~\$7+	~\$3-5, EU regions	~€4-5 flat (CX22, Falkenstein)	Same + nicer deploys
Long-running pipeline jobs (30-60s)	Poor fit	OK	OK	OK	Native	Native
GDPR narrative	US platform	US	US	US company, EU regions	German company, German datacenter	Same
Portfolio signal	"Deployed to Vercel"	Low	Low	Medium	"Dockerized, reverse-proxied, self-operated"	Same, slightly less hardcore

Choice: Hetzner CX22 (2 vCPU / 4 GB / 40 GB, ~€4/month) in Falkenstein, Docker Compose (frontend, API, Postgres, Uptime Kuma), Caddy for TLS + rate limiting, deploys via GitHub Actions over SSH. This is the cheapest production-ready option, the only one where "your document never leaves Germany except for the disclosed AI subprocessor" is literally true, and it demonstrates exactly the ops competence (Docker, reverse proxy, backups, monitoring) that differentiates a PM-turned-builder from a template deployer. Optionally add **Coolify** (self-hosted, free) for push-to-deploy comfort — it's a good middle

ground if raw Compose feels slow. A hybrid (Next.js on Vercel free + API on Hetzner) is defensible for frontend DX but splits the data-locality story; not worth it for a two-page app.

3.10 Observability

MVP includes exactly three things: **Sentry** (free tier: 5K errors/month, EU data residency selectable — wire both frontend and FastAPI), **Uptime Kuma** (self-hosted, free, pings the public URL and alerts by email/Telegram), and **structured JSON logs** (loguru/structlog) with per-request pipeline timings and per-letter AI cost written to Postgres — that last table is your FinOps dashboard for free. Defer: Grafana/Prometheus (ops theater at one-box scale), full OpenTelemetry (add spans later if the pipeline grows), PostHog EU (add at launch+1 month when funnel questions become real — free tier is generous). Logtail/Better Stack: fine, unnecessary.

3.11 Testing

Per the execution plan, formalized: **pytest** (+pytest-asyncio, httpx) for unit/integration; **Playwright** for the E2E happy path; **the custom eval harness as the centerpiece** — golden letters as fixtures, per-field precision/recall scorecard, fast subset in CI, full run at sprint boundaries and on any prompt/model/OCR change. OCR regression testing rides the same harness: fixture images → adapter → assert normalized output stability, so a provider swap or version bump is caught by CI, not by users. Tools like promptfoo can layer on later; a bespoke harness fitted to your schemas is more valuable in both function and signal.

3.12 CI/CD — GitHub Actions

Free for public repos, zero alternatives worth evaluating at this scale (keep GitLab CI/Woodpecker in mind only if the repo ever goes private-heavy). Pipeline: **PR** → ruff + mypy + eslint → pytest → eval fast-subset (5 letters, accuracy floor gate) → Playwright smoke → Docker build. **Merge to main** → build + push image → SSH deploy to Hetzner → health-check → Sentry release tag. Secrets in GitHub Actions secrets + a `.env` on the server; never in the repo. The eval-gate-in-CI step is the professional flourish reviewers won't expect from a portfolio project.

3.13 Security & GDPR checklist

Uploads: MIME + magic-byte validation, 20 MB cap, image re-encode on ingest (strips embedded payloads), no user-supplied filenames touching disk paths. Secrets: env-injected, rotated if ever exposed, separate keys per environment. API: rate limiting at Caddy + per-session app limits, CORS locked to the domain, request size caps. Encryption: TLS via Caddy; disk-level encryption on the VPS volume; Postgres password auth on the internal network only. GDPR: EU/German processing at every layer, subprocessors (OCR + LLM) named on the privacy page, no-training API configurations, deletion verified at storage layer, 24 h auto-purge job with logged executions, data-minimal by design (no accounts). Audit logging: append-only table of pipeline events per analysis (created, processed, purged/deleted) — cheap now, gold for any future B2B conversation.

4. Cost Model

Assumptions: 3 letters per active user per month, 3 pages per letter, Haiku-class LLM with caching, Azure Read OCR. Figures are estimates for planning, not quotes.

Cost line	100 users	1,000 users	10,000 users
Letters / pages per month	300 / 900	3,000 / 9,000	30,000 / 90,000
Hosting (Hetzner)	€4 (CX22)	€8-15 (CX32 + backups)	€40-80 (2 nodes + LB or CAX41)
OCR	€1.30	€13	€130 (Azure Read) / ~€90 (Mistral batch)
LLM	€6-10	€60-100 (→ €40 with routing)	€600-1,000 (→ €350-500 with routing + batch + caching)
Storage	€0 (local)	€5 (object storage)	€10-15
Monitoring	€0	€0	€25 (Sentry Team)
Database	€0	€0	€25 (managed Postgres or bigger node)
Total	~€12-16	~€90-135	~€830-1,275
Unit cost per letter	~€0.04	~€0.04	~€0.03

The FinOps punchline: costs scale almost perfectly linearly with letters, and the unit cost (~4 cents) is 50-100× below any plausible price point (€2-5 per case). There is no scale at which this architecture's AI costs break the business — the binding constraint is acquisition, never inference. The upgrade triggers are operational, not financial: move Postgres to managed at ~5K users (backup risk), add a second app node at sustained CPU >70%, revisit self-hosted LLM only if an enterprise/B2B deal demands zero third-party processing.

5. Development Environment Setup (Day 1 checklist)

1. Repo: monorepo (`web/`) (Next.js + TS) and (`api/`) (FastAPI), (`docker-compose.yml`) at root (`api`, `web`, `postgres`, `uptime-kuma`), (`.env.example`) committed.
2. Python via (`uv`); Node via (`pnpm`); pre-commit hooks: `ruff`, `mypy`, `eslint`, `prettier`.
3. Local Tesseract behind the OCR adapter so dev/CI never bill a cloud API; Azure/Mistral keys only in staging/prod env.

- 4. `make dev` (compose up + hot reload), `make test`, `make eval` — three commands, documented in README from day one.
- 5. Hetzner CX22 provisioned via cloud-init: Docker, Caddy, UFW (22/80/443), fail2ban, unattended-upgrades, non-root deploy user, nightly `pg_dump` cron to off-box storage.

6. Production Deployment Plan

Day 1: walking skeleton deployed (per execution plan) — DNS → Caddy → containers, TLS automatic. Every merge to main ships through the Actions pipeline with a post-deploy health check and automatic Sentry release tagging. Rollback = redeploy previous image tag (kept for last 10 builds). Backups: nightly DB dump + weekly restore drill (a restore you've never tested is not a backup). Launch hardening per the Day-24 checklist in the execution plan.

7. Future Upgrade Path (write it down, don't build it)

Trigger	Change
Accounts ship	Better Auth in Postgres; storage adapter flips to Hetzner Object Storage
5K+ users	Managed Postgres (or Supabase EU); PostHog EU analytics
Sustained load	Second Hetzner node + LB; queue (arq/Celery + Redis) for pipeline jobs
B2B/enterprise deal	Dedicated tenant deployments; self-hosted LLM evaluation (vLLM on rented EU GPU); SOC2-style audit log export
OCR/LLM market shifts	Re-run the bake-off through the adapters; swap via config + eval verification

8. Final Word

This stack optimizes for the three audiences that matter, in order: **users** (fast, private, works on a phone), **hiring managers** (Python + Pydantic pipeline, eval harness in CI, Docker on a self-operated German VPS, honest cost model), and **future-you** (two abstraction seams exactly where markets are volatile, boring choices everywhere else). Total spend under €15/month until real users force a better problem to have.